Rod Ciombor
3/6/26
CIS-2532-NET01
Dr. Sheikh Shamsuddin
Week 6 Lab

Functions library contents
Here is a list of the contents in **week6hw.py**

In [ ]:
```python
# Author:       Rod Ciombor
# Date:         03/06/2026
# Instructor:   Dr. Sheikh Shamsuddin
# Class:        CIS-2532-NET01

import re
import os

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))

def convertToLDAP(inputFileName):
    '''
    Name:
    convertToLDAP()

    Parameters:
    inputFileName (str): Name of file to be read from (not including directory)

    Returns:
    Nothing

    Descr:
    Serves as a main/controller function.
    Gets the contents from a given input file,
    parses each line into LDAP format,
    then writes the data to a new file.
    '''
    contents = getFileContents(inputFileName)

    if contents is False:
        print('Exiting program')
        return
    elif len(contents) == 0:
        print('No file contents detected')
        print('Exiting program')
        return
    else:
        writeContentsToFile('week6output.txt', contents)

def getFileContents(fileName):
    '''
    Name:
```

```
    getFileContents()

    Parameters:
    fileName (str): Name of file to be read from (not including directory)

    Returns:
    list: A list containing each line of the file as a string
    False: Returned if the file cannot be opened or another error occurs

    Descr:
    Builds the full file path using the script directory and the provided file name
    Attempts to open the file and read each line into a list, which is then returne
    If the file is not found or another exception occurs during processing,
    an error message is printed and False is returned.
    '''

    contents = []

    try:
        #Build filepath
        filePath = os.path.join(SCRIPT_DIR, fileName)

        #Read lines to list and return contents
        with open(filePath, 'r') as file:
            for line in file:
                contents.append(line)
        return contents

    except FileNotFoundError:
        print('Error! File not found!')
        return False

    except Exception as e:
        print('Error!  Something went wrong!')
        print(e)
        return False

def parseToLDAP(line):
    '''
    Name:
    parseToLDAP()

    Parameters:
    line (str): A colon-delimited string containing username, first name, last name

    Returns:
    list: A list of formatted LDAP record strings

    Descr:
    Splits the input line using ':' as the delimiter and removes any
    leading or trailing whitespace from each part.

    The function expects exactly four fields:
    username, first name, last name, and phone number.

    If the line does not contain exactly four elements, a ValueError is raised.
```

```
    If the input is valid, the function constructs and returns a list containing
    the formatted LDAP fields (dn, cn, sn, and telephoneNumber).
    '''
    output = []

    #Split line at delimiter and sanitize
    parts = line.split(':')
    parts = [part.strip() for part in parts]

    #Check if parts length == 4
    if len(parts) != 4:
        raise ValueError
    else:
        #Build output list
        output.append(createDnStr(parts[0]))
        output.append(createCnStr(parts[1], parts[2]))
        output.append(createSnStr(parts[2]))
        output.append(createLDAPPhoneStr(parts[3]))
    return output

def createDnStr(username):
    '''
    Name:
    createDnStr()

    Parameters:
    username (str): The username used for the LDAP distinguished name

    Returns:
    str: Formatted LDAP distinguished name string

    Descr:
    Creates and returns the LDAP distinguished name (dn) string using
    the provided username and the fixed domain components
    "dc=example, dc=com".
    '''
    return 'dn: uid={}, dc=example, dc=com'.format(username)

def createCnStr(first, last):
    '''
    Name:
    createCnStr()

    Parameters:
    first (str): First name of the user
    last (str): Last name of the user

    Returns:
    str: Formatted LDAP common name string

    Descr:
    Combines the first and last name values into a formatted LDAP
    common name (cn) string.
    '''
    return 'cn: {} {}'.format(first, last)
```

```
def createSnStr(lastName):
    '''
    Name:
    createSnStr()

    Parameters:
    lastName (str): Last name of the user

    Returns:
    str: Formatted LDAP surname string

    Descr:
    Creates and returns the LDAP surname (sn) field using the
    provided last name.
    '''
    return 'sn: ' + lastName

def createLDAPPhoneStr(phone):
    '''
    Name:
    createLDAPPhoneStr()

    Parameters:
    phone (str): Phone number associated with the user

    Returns:
    str: Formatted LDAP telephone number string

    Descr:
    Creates and returns the LDAP telephoneNumber field using
    the provided phone number.
    '''
    return 'telephoneNumber: ' + phone

def writeContentsToFile(outputFileName, contents):
    '''
    Name:
    writeContentsToFile()

    Parameters:
    outputFileName (str): Name of the file where the LDAP data will be written
    contents (list): List of strings representing lines read from the input file

    Returns:
    None

    Descr:
    Creates or overwrites the specified output file and writes LDAP-formatted
    records to it.

    Each line from the contents list is parsed using the parseToLDAP() function.
    If a line is incorrectly formatted and raises a ValueError,
    an error message is printed indicating the line number and the line is skipped.

    For valid lines, the function prints the formatted LDAP fields to the
```

```
    console and writes them to the output file.

    Each LDAP record is separated by a blank line in both the console output and th

    When processing is complete, a confirmation message is displayed.
    '''

    print('LDAP Data:')
    print('----------------------------------')
    print()

    lineNum = 1
    filePath = os.path.join(SCRIPT_DIR, outputFileName)

    #Create/open output file
    with open(filePath, 'w') as file:

        #Parse contents from input file to LDAP format
        for c in contents:
            try:
                output = parseToLDAP(c)
            except ValueError:
                print(f'Error! Incorrectly formed data on line {lineNum}')
                print()
                continue
            #Write LDAP data to output file and print to console
            for o in output:
                print(o)
                file.write(o + '\n')

            print()
            file.write('\n')
            lineNum += 1

        print('Data saved to week6output.txt')

def handleWebFormPhone():
    '''
    Name:
    handleWebFormPhone()

    Parameters:
    None

    Returns:
    Nothing

    Descr:
    This function prompts the user to enter a phone number,
    then cleans and validates the user input.

    If an invalid phone number was entered, it prompts the user to try again.

    If a valid phone number was entered,
    it displays the phone number in a standardized format.
```

```
    It then asks the user if they would like to enter another phone number.
    This process will repeat for as long as the user enters 'Y' or 'y'
    '''

    #Begin loop asking user to enter phone
    choice = 'Y'

    while choice == 'Y':
        #Get user input
        phone = str(input('Please enter a ten digit phone number: ')).strip()

        #Check all 4 patterns
        if not validatePhonePattern(phone):
            print('Invalid phone number entered!  Please try again')
            continue
        else:
            #Strip all non-numerical characters
            phone = cleanPhoneInput(phone)
            #Check must be ten digits long
            if not validatePhoneLength(phone):
                print('Phone number must be 10 digits!  Please try again')
                continue
            else:
                print(f'Formatted phone number: {buildPhoneOutput(phone)}')
                print()
                print('Would you like to enter another phone number?')
                choice = str(input('Enter Y for Yes or N for No: ')).upper()
                print()

    print('Thank You!')

def validatePhonePattern(phone):
    '''
    Name:
    validatePhonePattern()

    Parameters:
    phone (str): Phone number string to validate

    Returns:
    bool: True if phone matches one of accepted patterns, otherwise False.

    Descr:
    Checks whether a phone number matches one of several accepted formats.

    Accepted formats include the following:
    555-555-5555
    (555) 5555555
    (555) 555 5555
    5555555555
    '''
    patterns = [
        r'^\d{3}-\d{3}-\d{4}$', #555-555-5555
        r'^\(\d{3}\) \d{7}$', #(555) 5555555
        r'^\(\d{3}\) \d{3} \d{4}$', #(555) 555 5555
        r'^\d{10}$' #5555555555
```

```
    ]
    for pattern in patterns:
        if bool(re.match(pattern, phone)):
            return True

    return False

def cleanPhoneInput(phone):
    '''
    Name:
    cleanPhoneInput()

    Parameters:
    phone (str): Phone number associated with the user

    Returns:
    str: A string containing only the numeric digits from the input.

    Descr:
    Removes all non-numeric characters from a phone number string.
    '''
    return re.sub(r'[^0-9]', '', phone)

def validatePhoneLength(phone):
    '''
    Name:
    validatePhoneLength()

    Parameters:
    phone (str): Cleaned phone number that should contain only digits

    Returns:
    bool: True if phone has exactly 10 digits, otherwise False.

    Descr:
    Checks whether a phone number contains exactly 10 digits
    '''
    return len(phone) == 10

def buildPhoneOutput(phone):
    '''
    Name:
    buildPhoneOutput()

    Parameters:
    phone (str): Cleaned phone number that should contain only digits

    Returns:
    str: Formatted phone number string.

    Descr:
    Formats a 10-digit phone number into a standardized display format.
    '''
    return "({}) {} {}".format(phone[:3], phone[3:6], phone[-4:])
```

Question 1

The following is the output for Question 1, which reads a file and converts the content to LDAP format.

Part One: Test error messages for file I/O issues

- Test 1: Try to open missing file.
- Test 2: Try to open invalid file (test generic except block).
- Test 3: Open empty file.

In [4]:
```
import week6hw as lib

lib.convertToLDAP('somefile.txt')
print()
lib.convertToLDAP('.')
print()
lib.convertToLDAP('week6input_empty.txt')
print()
```

```
Error! File not found!
Exiting program

Error!  Something went wrong!
[Errno 13] Permission denied: 'C:\\Users\\16308\\.'
Exiting program

No file contents detected
Exiting program
```

Part Two: Demonstrate reading from valid file with valid data

Contents of week6input.txt

rciombor:Rod:Ciombor:630-853-3194 ckrajniak:Chris:Krajniak:630-743-1677 dnovak:Dan:Novak:630-779-4740
jtiscareno:Joe:Tiscareno:630-901-1645 rmikula:Robin:Mikula:630-391-8306

Show output with valid data

In [5]:
```
import week6hw as lib

lib.convertToLDAP('week6input.txt')
print()
```

```
LDAP Data:
----------------------------------

dn: uid=rciombor, dc=example, dc=com
cn: Rod Ciombor
sn: Ciombor
telephoneNumber: 630-853-3194

dn: uid=ckrajniak, dc=example, dc=com
cn: Chris Krajniak
sn: Krajniak
telephoneNumber: 630-743-1677

dn: uid=dnovak, dc=example, dc=com
cn: Dan Novak
sn: Novak
telephoneNumber: 630-779-4740

dn: uid=jtiscareno, dc=example, dc=com
cn: Joe Tiscareno
sn: Tiscareno
telephoneNumber: 630-901-1645

dn: uid=rmikula, dc=example, dc=com
cn: Robin Mikula
sn: Mikula
telephoneNumber: 630-391-8306

Data saved to week6output.txt
```

Contents of week6output.txt

dn: uid=rciombor, dc=example, dc=com cn: Rod Ciombor sn: Ciombor telephoneNumber: 630-853-3194 dn: uid=ckrajniak, dc=example, dc=com cn: Chris Krajniak sn: Krajniak telephoneNumber: 630-743-1677 dn: uid=dnovak, dc=example, dc=com cn: Dan Novak sn: Novak telephoneNumber: 630-779-4740 dn: uid=jtiscareno, dc=example, dc=com cn: Joe Tiscareno sn: Tiscareno telephoneNumber: 630-901-1645 dn: uid=rmikula, dc=example, dc=com cn: Robin Mikula sn: Mikula telephoneNumber: 630-391-8306

Part Three: Demonstrate reading from valid file with invalid data

Contents of week6input_bad_data.txt

rciombor:Rod:Ciombor:630-853-3194 ckrajniak:Chris:Krajniak:630-743-1677 dnovak:DanNovak:630-779-4740 jtiscareno:Joe:Tiscareno:630-901-1645 rmikula:Robin:Mikula:630-391-8306

Show output with invalid data

In [6]:
```
import week6hw as lib

lib.convertToLDAP('week6input_bad_data.txt')
print()
```

```
LDAP Data:
----------------------------------

dn: uid=rciombor, dc=example, dc=com
cn: Rod Ciombor
sn: Ciombor
telephoneNumber: 630-853-3194

dn: uid=ckrajniak, dc=example, dc=com
cn: Chris Krajniak
sn: Krajniak
telephoneNumber: 630-743-1677

Error! Incorrectly formed data on line 3

dn: uid=jtiscareno, dc=example, dc=com
cn: Joe Tiscareno
sn: Tiscareno
telephoneNumber: 630-901-1645

dn: uid=rmikula, dc=example, dc=com
cn: Robin Mikula
sn: Mikula
telephoneNumber: 630-391-8306

Data saved to week6output.txt
```

Contents of week6output.txt excluding bad data

dn: uid=rciombor, dc=example, dc=com cn: Rod Ciombor sn: Ciombor telephoneNumber: 630-853-3194 dn: uid=ckrajniak, dc=example, dc=com cn: Chris Krajniak sn: Krajniak telephoneNumber: 630-743-1677 dn: uid=jtiscareno, dc=example, dc=com cn: Joe Tiscareno sn: Tiscareno telephoneNumber: 630-901-1645 dn: uid=rmikula, dc=example, dc=com cn: Robin Mikula sn: Mikula telephoneNumber: 630-391-8306

Question 2

The following is the output for Question 2, prompts a user to enter a phone number, validates it, then displays it in a standardized format.

In [7]:
```
import week6hw as lib

lib.handleWebFormPhone()
```

```
Invalid phone number entered!  Please try again
Invalid phone number entered!  Please try again
Invalid phone number entered!  Please try again
Formatted phone number: (630) 853 3194

Would you like to enter another phone number?
Formatted phone number: (630) 743 1677

Would you like to enter another phone number?
```

```
Formatted phone number: (630) 901 1645

Would you like to enter another phone number?
Formatted phone number: (630) 391 8306

Would you like to enter another phone number?
Thank You!
```

In [ ]: